REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

N/A

HOMEWORK EXERCISES

1. ATMEL STUDIO INSTALLATION

Successfully installed Microchip Studio 7.0! (See Appendix)

2. LEARNING HOW TO USE ATMEL STUDIO

Did exercises specified (See Appendix for required photos)

PSEUDOCODE/FLOWCHARTS

SECTION 1: Provided Code

PSEUDOCODE:

Include ATxmega1228A1Udef.inc file

Initialize program constants/variables

Org 0

Short jump to MAIN FUNCTION

ORG to $100

MAIN FUNCTION:

; LED is PORTD_DIRSET

Set LED to BIT456

Set LED to RED

Set LED to GREEN

Set LED to BLUE

Set LED to WHITE

Set LED to PINK/MAGENTA

Set LED to BLACK

Clear BIT6 from LED (turning ~BLUE on from active-low behavior)

Set BIT6 from LED (turning BLACK from active-low behavior)

LOOP FUNCTION:

Toggle LED (with stored BIT6, turning on and off forever.)

Short jump to LOOP FUNCTION

PROGRAM CODE

SECTION 1: Provided Code

```
/****
 * GPIO_Output.asm
 *
 *  Modified: 29 Jan 2026
 *  Author: Dr. Schwartz
 *  Student: Evan Joseph Baesler

 This program shows how to initialize a GPIO port on the Atmel
 (Ports D for this example) and demonstrates various ways to write to
 a GPIO port. The output will blink LEDs at the bottom left of the
 uPAD, labeled D4. PortD4, PortD5, and PortD6 are the red, green,
 and blue LEDs, respectively. Note that these LEDs are active-low.

 PortC, not used in this example, has eight active-low LEDs on the
 Switch/LED backpack. It also has eight switch circuits on PortA that each
 use pull-up resistors.

 It should be noted that PortD2 (an XMEGA input) and PortD3 (an XMEGA output)
 are used for the EDBG (embedded debugger) and outputs to these pins should
 be avoided. These pins are intialized in the bootloader code that runs before
 any other code.
****/

;Definitions for all the registers in the processor. ALWAYS REQUIRED.
;View the contents of this file in the Processor "Solution Explorer"
;   window under "Dependencies"
.include "ATxmega128A1Udef.inc"

.equ   BIT4   = 0b00010000
.equ   INV4   = 0b11101111
.equ   RED = INV4
.equ   BIT5 = 0b00100000
.equ   INV5 = ~BIT5
.equ   GREEN = ~(BIT5)
.equ   BIT6 = 0x40
.equ   BLUE = ~(BIT6)
.equ   BIT456 = 0x70
.equ   WHITE = ~(BIT456)
.equ   BIT64 = 0x50
.equ   PINK = ~(BIT64)
.equ   BLACK = 0xFF

; Any necessary initializations for SRAM in data space?

.ORG 0
;Skip address through 0x00FD (interrupt vectors)
       rjmp MAIN

;Start program at 0x0100 so we don't overwrite vectors (0-0x00FD)
.ORG 0x0100
MAIN:

; initialize the data direction of the three LEDs as outputs (PD4-6)
       ldi r16, BIT456
       sts PORTD_DIRSET, r16
; Notice that the 3 LEDs (RED, GREEN, and BLUE) are all now on, creating white
```

```
; This is because these LEDs are active-low & the default value for outputs is low

; The following code shows different ways to write to the GPIO pins.

; Turn on each of the primary colored LEDs in turn, then use some combinations
       ; These instructions sends the value in r16 to the PORTD pins.
       ; Since the LEDs are wired as active-low,
       ;    r16 = RED = 0xEF = 0b1110 1111 will turn the RED LED on.
       ldi    r16, RED
       sts PORTD_OUT, r16

; Turn on the green LED (only)
       ldi    r16, GREEN
       sts PORTD_OUT, r16
; The below line could replace the above line if not for PortD3      was not an output
;      sts PORTD_OUT, r16

; Turn on the blue LED (only)
       ldi    r16, BLUE
       sts PORTD_OUT, r16

; Turn on all of the colored LEDs to make white
       ldi    r16, WHITE
       sts PORTD_OUT, r16

; Turn on blue and red to make a pink (magenta)
       ldi    r16, PINK
       sts PORTD_OUT, r16

; Turn off all of the LEDs (i.e., make it black)
       ldi    r16, BLACK
       sts PORTD_OUT, r16

; Turn on the blue LED (only)
; Notice that the OUTCLR makes the (blue) LED go on,
;   since the LED is active-low
       ldi    r16, BIT6                  ; BIT6 = ~BLUE
       sts PORTD_OUTCLR, r16

; Turn off the blue LED (i.e., make BIT6 a 1, using OUTSET)
; Notice that the OUTSET makes the (blue) LED go off,
;  since the LED is active-low
       ldi    r16, BIT6
       sts PORTD_OUTSET, r16

; Notice that the OUTTGL toggles the value of a PORT pin
;   (in this case the BLUE LED)
LOOP:
       sts PORTD_OUTTGL, r16
       rjmp LOOP                          ;repeat forever!
```

APPENDIX

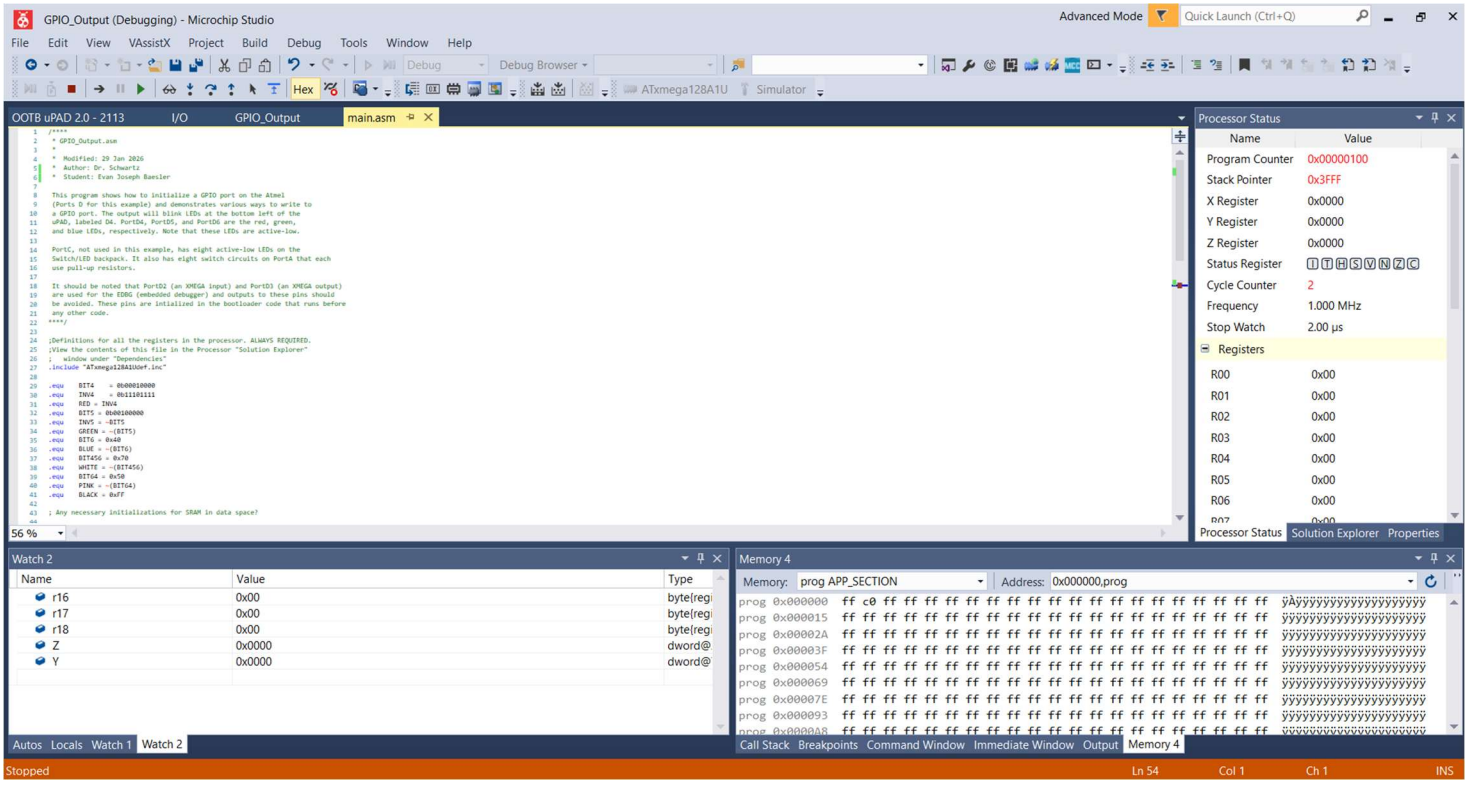

Figure 1: Microchip Studio simulating the required code, name shown below Author line.

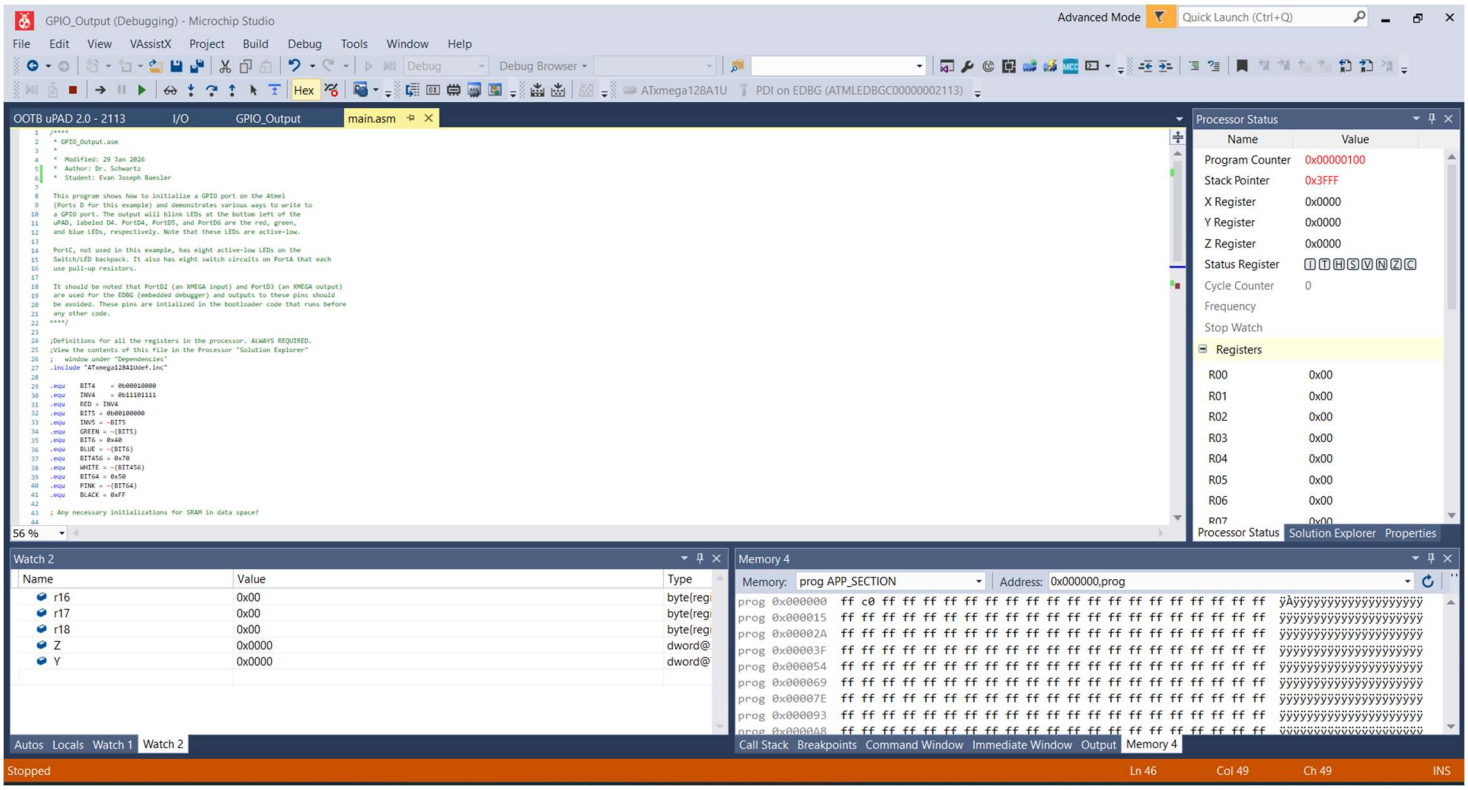

Figure 2: Microchip Studio running the required code on our OOTB board, name shown below Author line.